sslOpenCrypt

The Complete Guide

Encryption · Signing · PKI · Digital Certificates

India DSC · Aadhaar eSign · DigiLocker · Post-Quantum Cryptography

First Edition	License	Platform
2025	GPL v3 — Open Source	Linux · macOS · Windows

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
This book explains cryptography — the science of secret messages — from absolute basics to deep technical detail. Every topic is explained four times at four different levels. Skip to the level that suits you. The software on the included pen drive does everything described in this book, with a live command display so you can learn as you go.

tctech.co.in
This book and the sslOpenCrypt software are free and open-source (GPL v3)

sslOpenCrypt — The Complete Guide

First Edition, 2025

Written and typeset using sslOpenCrypt document tools and Claude AI.

Source code, document source, and all content licensed under the GNU General Public License version 3 (GPL v3). You are free to copy, modify, and redistribute this book and the software, provided you preserve the licence and attribution.

Regulatory references current as of April 2025: IT Act 2000, CCA-IVG July 2024, MeitY eSign API v3.2, NIST FIPS 203/204/205 (2024).

Publisher	tctech.co.in — Independent technical publishing
Author	Rajiv, tctech.co.in
Software	sslOpenCrypt (GPL v3) — github.com/tctech/sslopencrypt [planned]
Typeset	A4 portrait, Arial body 11pt, Courier New code 9.5pt
Pen drive	sslOpenCrypt v0.x AppImage (Linux), .dmg (macOS), .exe installer (Windows)
Contact	tctech.co.in | feedback via GitHub Issues

DISCLAIMER: This book is provided for educational purposes. Cryptographic regulations, government portal requirements, and algorithm recommendations change over time. Always verify current CCA guidelines at cca.gov.in and NIST publications at csrc.nist.gov before making compliance decisions. The authors are not liable for any consequences arising from the use of information in this book.

Foreword

Cryptography is no longer the exclusive domain of intelligence agencies, defence contractors, and academic researchers. In the decade since India enacted its Digital India initiative, and in the years since NIST began standardising post-quantum algorithms, cryptographic operations have become part of everyday life — filing a GST return, signing a company resolution, accessing DigiLocker, buying a domain certificate, or authenticating a firmware update.

Yet the tools that perform these operations have remained stubbornly opaque. A developer working with OpenSSL for the first time faces a terse command-line interface, a man page the size of a small novel, and error messages that assume deep knowledge of ASN.1 and RFC 5280. At the other end of the spectrum, a director needing to sign an MCA21 filing is handed a USB token, a proprietary driver CD of uncertain provenance, and instructions that assume a working Java installation from 2012.

sslOpenCrypt is an attempt to close both gaps simultaneously. It is a graphical application — built on PyQt6, calling the system OpenSSL binary, and displaying every command it runs — that is useful to a school student signing their first document and to a security engineer auditing a certificate chain. Every operation has a four-level help system: ELI5 for the curious beginner, Beginner for the non-technical user, Intermediate for the developer, and Expert for the security professional.

This book is that help system in printed form. It is also the product specification, the cryptographic reference manual, and the integration guide — for the Indian Digital Signature Certificate ecosystem, for Aadhaar eSign, for DigiLocker, and for the coming generation of post-quantum algorithms. It ships with the sslOpenCrypt software on the included pen drive.

Every chapter of the reference sections is written at all four levels. Every code example is a real OpenSSL command you can run. Every Indian government portal reference has been verified against current CCA guidelines. The software on the pen drive does what the book describes.

We hope this book and software help make cryptography — which is really just mathematics in service of trust — accessible to everyone who needs it.

Rajiv Tyagi
tctech.co.in
Meerut, India — April 2026

Table of Contents

PART I — Product Specification

Section 1	Executive Summary — value propositions, design philosophy
Section 2	Product Identity — name, stack, licence, platform targets
Section 3	Target Audiences — five user profiles with concrete use cases
Section 4	Application Architecture — three-layer design, mode system, Command Console
Section 5	Functional Modules — nine core modules specified in full
Section 6	External Integrations — LibreOffice, Thunderbird, Gmail, file managers, VS Code
Section 7	Key Design Decisions — CLI vs library, PyQt6 rationale, security hardening
Section 8	Development Roadmap — v0.1 through v2.x milestones
Section 9	Open Questions — community discussion points
Section 10	Contributing & Repository Structure
Section 11	References & Related Projects
Section 12	Verbose Help System — four-tier help, error humanisation, UI integration

PART II — Cryptography Reference Manual

Each chapter is written at four levels: 🧒 ELI5 · 🌱 Beginner · ⚙️ Intermediate · 🔬 Expert

Chapter 1	Core Cryptographic Concepts — keys, CIA triad, computational security
Chapter 2	Hashing & Message Digests — SHA-256/512, SHA-3, BLAKE2, HMAC
Chapter 3	Symmetric Encryption — AES modes (GCM, CBC, CTR), ChaCha20-Poly1305
Chapter 4	Asymmetric / Public-Key Cryptography — RSA, ECDSA, Ed25519, key pairs
Chapter 5	Entropy, Randomness & Key Derivation — CSPRNG, PBKDF2, Argon2id, HKDF
Chapter 6	PKI & X.509 Certificates — CA hierarchy, CSRs, CRLs, OCSP, RFC 5280
Chapter 7	Digital Signatures & Signing Formats — CMS, CAdES, PAdES, RFC 3161
Chapter 8	S/MIME & Email Encryption — EnvelopedData, key agreement, vs PGP/GPG
Chapter 9	TLS & HTTPS Internals — TLS 1.3 handshake, key schedule, cipher suites
Chapter 10	OpenSSL CLI — complete subcommand reference, essential one-liners
Chapter 11	Post-Quantum Cryptography — ML-KEM, ML-DSA, SLH-DSA, OQS-OpenSSL

PART III — India Digital Signature Ecosystem

Chapter 12	Legal Foundation — IT Act 2000, CCA, India PKI hierarchy, July 2024 IVG
Chapter 13	Class 3 DSC USB Tokens — hardware, PKCS#11 drivers, Linux/macOS/Windows
Chapter 14	Aadhaar eSign — OTP/Biometric/Offline XML, ESP API, privacy considerations
Chapter 15	DigiLocker Integration — OAuth2 API, document fetch, verification, sharing
Chapter 16	Government Portals — MCA21, GSTN, DGFT, Income Tax, EPFO, NeSL, eCourts
Chapter 17	sslOpenCrypt India DSC Module — spec for Module 10, portal integrations roadmap

Appendices

Appendix A	Algorithm Quick-Reference — security levels, deprecation schedule
Appendix B	Glossary — 60+ cryptographic and regulatory terms defined at four levels
Appendix C	India DSC Certifying Authorities — licensed CA list, contact reference
Appendix D	Pen Drive Contents — software, source code, reference files, CCA trust chain

Part I

Product Specification

sslOpenCrypt — Design, Architecture & Feature Specification

§ 1

Executive Summary

What sslOpenCrypt is, why it exists, and what makes it different

Mission

sslOpenCrypt is a cross-platform, open-source graphical application that brings the full power of OpenSSL to every user — from a student learning cryptography for the first time, to a company director signing MCA21 filings, to a security engineer managing a private certificate authority. It is built for India as much as for the world.

Its defining philosophy is transparency: every operation is shown as the equivalent openssl command in a live Command Console. Using sslOpenCrypt is simultaneously using OpenSSL — and learning it.

Core Value Propositions

Pillar	What it means	How sslOpenCrypt delivers it
Transparency	Every operation is auditable	Live Command Console shows real openssl commands before and after execution
Accessibility	Works for beginners and experts	Four-tier help system (ELI5/Beginner/Intermediate/Expert) on every feature
India-ready	Native Indian DSC & eSign support	USB token PKCS#11 integration, Aadhaar eSign API, DigiLocker OAuth2
Educational	Teaches cryptography, not just uses it	Interactive tutorials, algorithm animations, Command Explorer, lab reports
Open Source	GPL v3 — freely auditable and forkable	GitHub repository, ReadTheDocs, community plugin system
Portable	Ships on a pen drive	AppImage, .dmg, NSIS installer — no installation required for AppImage

§ 2

Product Identity

Name, stack, licence, and platform decisions

Identity at a Glance

App name	sslOpenCrypt
Licence	GNU GPL v3
Language	Python 3.10+
GUI framework	PyQt6 (Qt 6.x)
Backend	openssl CLI (subprocess) + Python cryptography library (parsing/validation)
Platforms	Linux (primary — AppImage), macOS (.dmg), Windows (NSIS installer)
Pen drive	AppImage runs without installation on any Linux distro from the drive directly
Repository	GitHub (primary) + GitLab mirror
Docs	ReadTheDocs + in-app four-tier help

Why These Technology Choices

openssl CLI (not library only)	Every operation displays the real openssl command. Users learn the industry-standard tool, not a wrapper abstraction. Commands are copy-paste reproducible on any server.
PyQt6 (not GTK or Electron)	Single Python codebase runs on all three platforms. Native look-and-feel per OS. GPL v3 compatible. Lower memory footprint than Electron. Strong signal/slot model for async subprocess.
GPL v3	Ensures the application and all derivatives remain open-source. Aligns with the educational mission. Prevents proprietary forks from hiding modifications.
AppImage for Linux	Zero-install: the pen drive can be plugged into any Linux machine and sslOpenCrypt runs immediately. Ideal for workshops, training sessions, and air-gapped environments.

§ 3

Target Audiences

Five user profiles and their primary use cases

Audience	Primary use cases	DSC / eSign need	Key sslOpenCrypt modules
General / Non-technical users	Encrypt personal files, sign documents, verify downloads	Aadhaar eSign (no token needed)	Symmetric Enc, Signing, Hashing
Developers & sysadmins	Key generation, CA management, TLS inspection, batch signing, JWT/code signing	Class 3 DSC or Ed25519 key	PKI, Key Mgmt, TLS Advisor, CLI
Students & educators	Learn cryptography concepts interactively, generate lab reports, explore algorithms	Usually none — sandbox keys	Educational Hub, Command Explorer
Small businesses & NGOs	Sign contracts, manage internal CA, issue employee certs, S/MIME email	Class 3 DSC (Sign+Encrypt)	PKI, S/MIME, Document Signing
Makers & embedded engineers	Device identity certs, firmware signing, MQTT-TLS, key derivation for RP2350	Ed25519 or ECDSA P-256 soft key	Key Mgmt, PKI, Hashing, Batch CLI

§ 4

Application Architecture

Three-layer design, mode system, and the Command Console

Three-Layer Architecture

⚙️ **[INTERMEDIATE] Intermediate — Technical Background**

Layer 1 — UI (PyQt6): All widgets, dialogs, panels, animations
Layer 2 — Controller (Python): Business logic, parameter validation, state
Layer 3 — Execution: subprocess → openssl CLI + python cryptography (parsing)

The Execution Layer returns a structured result: command run, stdout/stderr, parsed output dict, success flag. This drives both the result panel AND the Command Console simultaneously — one operation, two views.

Four Operating Modes

Beginner Mode	Command Console hidden. Plain-English labels. Algorithm choices simplified to safe defaults. One-sentence tooltips on every field.
Expert Mode	Command Console visible and editable. All algorithm parameters exposed. Raw hex/ASN.1 output. Import/export any PEM/DER/PKCS format. Deprecated algorithms (MD5, SHA-1, RC4, 3DES): available in Expert Mode with amber-border alert, mandatory confirmation checkbox, Console # WARNING comment, and DEPRECATED_ALG audit-log flag.
Classroom Mode	Instructor-defined constraints. Session logging forced. Exportable HTML/PDF lab reports. Quiz mode with progress tracking.
Batch / CLI Mode	Headless execution via command-line arguments. JSON output. For CI pipelines and scripted bulk operations.

The Command Console — The Signature Feature

Every GUI action maps to one or more openssl commands, displayed in real time with syntax highlighting. Users can copy any command, edit and re-run it (Expert Mode), bookmark frequently used commands, and export the full session as a .sh or .bat script.

```
# Example — what the Console shows when generating an Ed25519 key pair:
openssl genpkey -algorithm ed25519 -out private.pem
openssl pkey -in private.pem -pubout -out public.pem

# With RSA-4096 + AES-256-CBC passphrase protection:
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 \
    -aes-256-cbc -out private.pem
openssl pkey -in private.pem -pubout -out public.pem
```

§ 5

Functional Modules

Nine core modules — summary specifications

#	Module	Core function	Key algorithms / formats
1	Key Management	Generate, inspect, convert, store asymmetric key pairs	RSA 1024–8192, ECDSA P-256/384/521, Ed25519, Ed448, X25519
2	Symmetric Encryption	Encrypt/decrypt files, text, folders with symmetric ciphers	AES-128/256-GCM/CBC/CTR/CCM, ChaCha20-Poly1305, PBKDF2/Argon2id KDF
3	Hashing	Compute and verify hashes; HMAC; batch file hashing	SHA-256/384/512, SHA-3, BLAKE2b/2s, HMAC-SHA-256
4	PKI & Certificates	CSR builder, CA manager, certificate inspector, CRL/OCSP	X.509 v3, RFC 5280, PKCS#10, PKCS#12, PEM/DER
5	Document & File Signing	Create and verify digital signatures on any file	CMS PKCS#7, CAdES-B/T/LT/LTA, PAdES, RFC 3161 timestamps
6	S/MIME & Email	Prepare keys for encrypted/signed email; contact book	CMS EnvelopedData, AES-256-CBC/GCM, RSA-OAEP, ECDH
7	Secure Random	Generate keys, passwords, primes, DH parameters	openssl rand, openssl prime, openssl dhparam, PBKDF2
8	TLS Advisor	Build cipher suite configs; inspect remote TLS servers	TLS 1.2/1.3, s_client, Mozilla Intermediate/Modern profiles
9	Educational Hub	Interactive tutorials, algorithm animations, command library	All algorithms; lab report export; quiz mode; glossary
10	India DSC & eSign	USB token signing, Aadhaar eSign, DigiLocker, portal integration	PKCS#11, CAdES/PAdES, eSign API v3.2, DigiLocker OAuth2
10	GnuPG / OpenPGP Integration	Wrap gpg2 for OpenPGP key gen, encrypt/decrypt, sign/verify	RSA-4096, Ed25519, Cv25519; SHA-256; GnuPG ≥2.2

§ 6

External Integrations

LibreOffice, Thunderbird, Gmail, file managers, VS Code

Integration	Mechanism	Operations	Target version
LibreOffice	Basic macro + IPC socket	Sign/verify Writer docs, sign Calc sheets, encrypt ODT/ODS	v1.0
Thunderbird	WebExtension (MV3) + native messaging	S/MIME sign/encrypt/verify in compose and read views	v1.1
Gmail (Chrome/Firefox)	Browser extension + native host	Encrypt body, sign, insert public key, decrypt received PGP/S/MIME	v1.2
Nautilus / Dolphin	Python extension / ServiceMenu .desktop	Encrypt, Decrypt, Sign, Verify, Hash from right-click menu	v1.0 / v1.1
VS Code	Extension + IPC	Sign/verify from Explorer; inspect .pem files with hover details	Community / v1.2
Git	Wrapper around gpg.program config	Sign commits and tags with sslOpenCrypt-managed keys	v1.x community

§ 7

Key Design Decisions

Rationale for major technical and product choices

openssl CLI not library	Displays the real-world commands users encounter on servers, in scripts, in documentation. Every operation is reproducible without sslOpenCrypt. The python cryptography library handles parsing and validation only.
PyQt6 not GTK/Electron	Single Python codebase across platforms. GPL v3 compatible. Native look per OS. 80–150 MB footprint vs 300 MB+ for Electron. Qt Designer for rapid UI work.
No telemetry	No crash reports, no analytics, no network calls except those the user explicitly initiates (OCSP check, TSA timestamp, TLS inspection). Private keys never touch the network.
Passphrase security	Passphrase fields use QLineEdit with EchoMode::Password. Content never written to undo history, clipboard, or swap. Clear-on-focus-loss option. Argon2id for key vault derivation.
Subprocess sanitisation	All openssl calls use list-form arguments (no shell=True). Arguments are validated before construction. openssl binary path is verified at startup against a stored SHA-256 hash.

§ 8

Development Roadmap

Milestones from alpha to post-quantum

Milestone	Focus areas	Platforms
v0.1 Alpha	Key Management, Symmetric Encryption, Hashing, Command Console, Beginner/Expert mode	Linux AppImage
v0.2 Alpha	PKI module, CA Manager, Certificate Inspector, TLS Inspector, File Signing	Linux AppImage
v0.3 Beta	S/MIME, Secure Random, TLS Advisor, Nautilus extension	Linux + macOS .dmg
v1.0 Stable	LibreOffice integration, Educational Hub, Classroom Mode, Lab Reports, India DSC Token Manager	All three + pen drive
v1.1	Thunderbird, OCSP/CRL UI, Batch CLI, Key Vault, Aadhaar eSign Client, India PKI Trust Store	All three
v1.2	Gmail extension, PAdES PDF signing, DigiLocker Connect, RFC 3161 TSA, VS Code extension	All three
v1.3	GSTN, DGFT, EPFO portal integrations, NeSL eSign, HSM/PKCS#11 deep support	All three
v2.x	Post-quantum keys (ML-KEM/ML-DSA/SLH-DSA via OQS-OpenSSL), Flutter companion app for Android/iOS (deferred from v1 core scope)	All three + Android

SECTION 8A — Embedded Systems Use Case

Secure OTA Firmware Signing — Raspberry Pi Pico 2W (RP2350)

Overview

The Raspberry Pi Pico 2W (RP2350) integrates hardware-accelerated cryptographic primitives: SHA-256, AES-128/256, and a hardware TRNG, with ARM Cortex-M33 TrustZone-M for optional secure/non-secure partitioning. sslOpenCrypt serves as the developer-side tooling for signed Over-the-Air (OTA) firmware delivery on this platform.

The OTA Signing Chain (ECDSA-SHA256)

Every firmware update is protected by an ECDSA-SHA256 digital signature using EC P-256 (prime256v1). The private key never leaves the developer machine; the corresponding public key is compiled into device firmware at build time and stored in flash. The Earle Philhower RP2350 Arduino core signing.py hook calls OpenSSL directly:

```
openssl ecparam -genkey -name prime256v1 -noout -out private.key
openssl ec -in private.key -pubout -out public.key
openssl dgst -sha256 -sign private.key -out sig.bin firmware.bin
# Signature (~72 bytes DER) appended to produce firmware.bin.signed
```

The device verifies the signature against the baked-in public key before writing any OTA update to flash. A mismatch results in immediate rejection — the device remains on the current firmware unchanged.

Role of sslOpenCrypt in the OTA Workflow

Key generation — Module 1 (Key Management) generates the EC P-256 key pair with GUI guidance, shows the exact openssl ecparam and openssl ec commands, and offers immediate import of private.key into the encrypted vault (Argon2id-derived AES-256-GCM).

Signature verification — Module 5 (Document & File Signing) verifies .bin.signed before distribution: it extracts the appended DER signature, runs openssl dgst -sha256 -verify, and shows pass/fail with key fingerprint and file sizes.

Key vault — private.key is stored in the vault between builds. Export before build, delete exported file after. Prevents accidental git commits of the private key.

Key rotation — when keys change (breaking: all devices need USB re-flash), sslOpenCrypt records the rotation in the audit log with timestamps and fingerprints.

Per-device licence binding — generate_uf2.py patches a device serial into the binary and re-signs it. sslOpenCrypt exports the vault key for the --privatekey argument and verifies each per-device .bin.signed output.

[BOX: CONFIGURATION] Recommended settings for embedded firmware signing:

Algorithm: ECDSA / EC P-256 (prime256v1) — compact 72-byte signatures
Digest: SHA-256 (required by signing.py hook)
Key protect: AES-256-CBC on PEM file; Argon2id in vault
Vault label: sslopencrypt-signing-[product]-[YYYY]
Do NOT use RSA — EC P-256 signatures are 72 bytes vs 256+ bytes for RSA-2048.
Do NOT substitute SHA-1 or MD5 — signing.py strictly requires SHA-256.

§ 9 Open Questions

Topics requiring community input before finalisation

1. App name — "sslOpenCrypt" is the working name. Community vote or founding team decision?
2. Internationalisation — English-only at launch, or invest in Qt Linguist i18n from v0.1? Priority languages: Hindi, Tamil, German, Portuguese, Japanese.

3. PGP/GPG — should sslOpenCrypt also wrap GnuPG for OpenPGP operations, or remain X.509/OpenSSL-only? A PGP module would significantly broaden the email encryption audience.
4. Key vault — compete with KeePassXC, or integrate with it via the KeePassXC secret service API?
5. Algorithm guardrails — allow deprecated algorithms (MD5, SHA-1, RC4) for legacy-system testing in Expert Mode, or block them entirely?
6. Funding model — OpenCollective donations, GitHub Sponsors, or a commercial Pro edition with enterprise CA features and support?
7. Mobile — Flutter companion app for Android/iOS: priority or distraction for a v1 core team?
8. Post-quantum timeline — OQS-OpenSSL integration in v1.x community branch, or wait for mainline OpenSSL PQC support (expected 2026+)?

§ 10

Contributing

Repository structure and areas needing help

```
sslopencrypt/
  core/           — subprocess wrapper, result types, temp-file manager
  modules/        — one subdirectory per functional module (01_keymgmt …
10_india_dsc)
  ui/             — PyQt6 widgets, main window, dialogs, Command Console
  integrations/   — LibreOffice macro, browser extensions, file manager plugins
  india/          — PKCS#11 token detection, eSign API client, DigiLocker OAuth2
  cli/            — headless batch mode entry point
  tests/          — pytest test suite (unit + integration)
  docs/           — ReadTheDocs source + this book in reStructuredText
  packaging/      — AppImage recipe, .dmg build, NSIS script, pendrive layout
  CONTRIBUTING.md
  LICENSE         — GPL v3
```

Area	Skills needed	Contact
UI/UX Design	Figma, Qt Designer, accessibility review, dark/light theme	GitHub Issues → label: ui
Cryptography review	Audit module designs for correctness; spot dangerous defaults	GitHub Issues → label: security
India DSC testing	Physical tokens (ePass2003, HYP2003, SafeNet), portal testing	GitHub Issues → label: india-dsc
Translations	Qt Linguist .ts files: Hindi, Tamil, German, Portuguese	GitHub Issues → label: i18n
Educational content	Tutorial scripts, animation storyboards, quiz questions	GitHub Issues → label: edu
Build & packaging	AppImage, Flatpak, Homebrew, Chocolatey, Snap, pen drive layout	GitHub Issues → label: packaging

§ 11

References

Standards, libraries, and related projects

OpenSSL	https://openssl.org — the underlying cryptographic engine
PyQt6	https://riverbankcomputing.com/software/pyqt/
python cryptography	https://cryptography.io — parsing and validation layer
endesive	https://github.com/m32/endesive — PAdES/CAdES Python library
OQS-OpenSSL	https://github.com/open-quantum-safe/openssl — post-quantum provider
Mozilla SSL Config	https://ssl-config.mozilla.org — TLS configuration reference
NIST PQC standards	FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA) — csrc.nist.gov
CCA India	https://cca.gov.in — Controller of Certifying Authorities, India PKI repository
MeitY eSign API	eSign API Specification v3.2 — cca.gov.in/sites/files/pdf/esign/eSign-APIv3.2.pdf
DigiLocker APIs	https://apisetu.gov.in/digilocker — Issuer and Requester API specifications
IT Act 2000	https://meity.gov.in — Information Technology Act 2000 and 2008 Amendment
RFC 5652	Cryptographic Message Syntax (CMS / PKCS#7)
RFC 8446	The Transport Layer Security (TLS) Protocol Version 1.3
RFC 5280	X.509 PKI Certificate and CRL Profile
RFC 3161	Time-Stamp Protocol (TSP)
RFC 9106	Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications

§ 12

Help System

Four-tier contextual help — design and topic coverage

sslOpenCrypt's help system provides four-tier explanations (ELI5 / Beginner / Intermediate / Expert) at every point in the UI — on every field, button, algorithm picker, and error message. The tiers are colour-coded and switchable at any time with Ctrl+Alt+T.

? icon on every field	Hover or click to open a four-tab help popover. All four tiers shown simultaneously.
F1 / Help key	Full help article for the active module at the last-used tier level.
Algorithm picker	Inline summary + Learn More link on every algorithm selection.
Command Console	Hover over any openssl argument for a Beginner + Expert tooltip.
Error messages	Every OpenSSL error intercepted: plain English + raw error shown side by side.
Educational Hub	Full four-tier articles browsable as a standalone reference — this book, inside the app.
Lab report export	Export any session as HTML or PDF with commands, outputs, and explanations.
Classroom mode	Instructor-defined constraints; quiz mode; progress tracking; forced session log.

Part II

Cryptography Reference Manual

Eleven chapters · Four explanation levels per topic · Full OpenSSL command examples

Ch. 1

Core Cryptographic Concepts

What is cryptography, and why does it matter?

What Is Cryptography?

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
Imagine you write a secret message to your friend, but you swap every letter for a different one using a code only you two know. That is cryptography — scrambling information so only the right person can read it. Modern computers do this billions of times a second.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
Cryptography is the science of securing information. It lets two parties communicate privately over a public channel (like the internet), prove that a message has not been tampered with, and verify who sent it. OpenSSL implements the mathematical algorithms that make all of this possible.

⚙️ **Intermediate — Some Technical Background**
Modern cryptography rests on three pillars: Confidentiality (only the intended recipient can read the message), Integrity (any tampering is detectable), and Authenticity (the sender's identity can be verified). A fourth property — Non-repudiation — means the sender cannot later deny having sent it. These four properties map directly to the primitives: encryption, hashing, MACs, and digital signatures respectively.

🔬 **[EXPERT] Expert — Deep Technical Detail**
Formal cryptography distinguishes between information-theoretic security (e.g., one-time pads — unconditionally secure) and computational security (security holds only if the adversary's compute power is bounded by a polynomial in the security parameter λ).
All practical schemes — RSA, AES, ECDSA — are computationally secure under specific hardness assumptions (integer factorisation, discrete logarithm, elliptic curve discrete logarithm). Reduction proofs tie the security of a scheme to the assumed hardness of one of these problems in the random oracle or standard model.

Keys: The Secret Ingredient

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
A key is like a password that locks and unlocks your scrambled message. The stronger and longer the password, the harder it is for someone to guess.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
A cryptographic key is a sequence of random bits used as input to an algorithm. Symmetric algorithms use one shared key for both encryption and decryption. Asymmetric algorithms use a mathematically linked pair: a public key (shareable) and a private key (secret). Key length is measured in bits — 256-bit AES, 4096-bit RSA.

⚙️ **Intermediate — Some Technical Background**
Key security is a function of both length and algorithm. A 128-bit symmetric key offers ~128 bits of security (exhaustive search). A 3072-bit RSA key offers only ~128 bits of equivalent security because RSA relies on factorisation, which is sub-exponential.

NIST SP 800-57 provides a table mapping symmetric and asymmetric key sizes to equivalent security levels. Keys must be generated from a cryptographically secure random source (CSPRNG) — never from time, PID, or user input without proper derivation.

🔬 **[EXPERT] Expert — Deep Technical Detail**
Key derivation functions (KDFs) such as HKDF (RFC 5869), PBKDF2 (RFC 8018), and Argon2id (RFC 9106) expand low-entropy inputs (passwords, shared secrets) into keying material of the required length. HKDF uses a two-step Extract-then-Expand paradigm over HMAC-SHA-256. Argon2id is memory-hard (configurable m KiB of RAM required per iteration), providing resistance against GPU/ASIC-based password cracking. Key encapsulation mechanisms (KEMs) such as HPKE (RFC 9180) and ML-KEM (FIPS 203) provide forward-secret key agreement without static private key exposure.

The CIA Triad in Cryptography

Confidentiality	Only the intended recipient can read the data. Achieved by: encryption (symmetric or asymmetric).
Integrity	The data has not been altered in transit. Achieved by: cryptographic hashes, MACs, and digital signatures.
Authenticity	The sender is who they claim to be. Achieved by: digital signatures, certificates, MACs.
Non-repudiation	The sender cannot deny having sent the message. Achieved by: asymmetric digital signatures (a MAC alone does not achieve this because both parties share the key).
Availability	Not a cryptographic property per se, but cryptography can protect availability against replay attacks and spoofing via nonces, timestamps, and sequence numbers.

Ch. 2

Hashing & Integrity

SHA, HMAC, BLAKE2 — fingerprinting data

What Is a Hash Function?

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
Imagine putting a book through a magic machine that always spits out the same short fingerprint — no matter how long the book is. Change even one letter, and the fingerprint is completely different. You cannot put the fingerprint back in to get the book.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
A hash function takes any input (a file, a password, a message) and produces a fixed-length output called a digest or hash. SHA-256 always outputs 256 bits (64 hex characters).
The same input always gives the same hash. Even a one-bit change causes a completely different hash (avalanche effect). You cannot reverse a hash to get the original input.
Use cases: file integrity checking, password storage (with salting), digital signatures.

⚙️ **Intermediate — Some Technical Background**
A secure hash function must satisfy: Pre-image resistance (given h, find m such that H(m)=h is computationally infeasible), Second pre-image resistance (given m1, find m2≠m1 with H(m1)=H(m2)), and Collision resistance (find any m1≠m2 with H(m1)=H(m2)).
SHA-1 is broken for collision resistance (SHAttered attack, 2017). MD5 is broken for both collision and second pre-image. Use SHA-256, SHA-384, SHA-512, SHA-3, or BLAKE2b.

🔬 **[EXPERT] Expert — Deep Technical Detail**
SHA-256 is built on the Merkle-Damgård construction with Davies-Meyer compression.
SHA-3 (Keccak) uses a sponge construction over a 1600-bit state — fundamentally different and immune to length-extension attacks that affect SHA-2. BLAKE2 is based on the ChaCha stream cipher core, offering comparable security to SHA-3 with higher throughput on 64-bit CPUs (~1 GB/s vs ~300 MB/s for SHA-256 without hardware acceleration).
For password storage, use a memory-hard function (Argon2id, scrypt, bcrypt) rather than a raw hash — these are designed to be slow and RAM-intensive to defeat GPU attacks.
BLAKE3 (not yet in OpenSSL mainline) further improves parallelism via a Merkle tree.

Algorithm Comparison

Algorithm	Output	Speed	Status	Use for
MD5	128-bit	Very fast	BROKEN — do not use for security	Legacy checksums only
SHA-1	160-bit	Fast	BROKEN (collisions)	Legacy only — avoid
SHA-256	256-bit	Fast	Secure	General purpose, TLS, signatures
SHA-384	384-bit	Medium	Secure	When 256-bit margin is not enough
SHA-512	512-bit	Medium	Secure	High-security contexts

SHA-3-256	256-bit	Moderate	Secure	Sponge-based alternative to SHA-2
BLAKE2b	512-bit	Very fast	Secure	Files, passwords (unsalted), streaming
BLAKE2s	256-bit	Fast	Secure	Embedded, 32-bit platforms

HMAC — Keyed Hashing for Message Authentication

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
A regular hash is like a fingerprint anyone can check. An HMAC is a fingerprint that only you and your friend can compute, because it uses a secret key too. So if someone fakes the message, they cannot fake the fingerprint without the key.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
HMAC (Hash-based Message Authentication Code) combines a hash function with a secret key. It proves both that the message has not changed AND that the sender knew the key. Common use: API request signing, JWT tokens (HS256 = HMAC-SHA-256), cookie integrity.

⚙️ **Intermediate — Some Technical Background**
HMAC(K, m) = H((K ⊕ opad) || H((K ⊕ ipad) || m)).
The double-hashing prevents length-extension attacks. HMAC is not a digital signature — it requires both parties to share the key, so it does not provide non-repudiation. HMAC-SHA-256 is the mandatory algorithm in TLS 1.2 PRF and many JWT implementations.

🔬 **[EXPERT] Expert — Deep Technical Detail**
HMAC is proven secure (as a PRF) whenever the underlying hash is a PRF, under the assumption that the compression function is a PRF. The ipad/opad construction (0x36/0x5C repeated) creates two independent derived keys. KMAC (from NIST SP 800-185) is the SHA-3-based keyed hash, avoiding the padding workaround needed for HMAC by using the sponge's capacity parameter. For authentication tags in AEAD ciphers, Poly1305 (used with ChaCha20) is a one-time MAC over GF(2^130-5) — faster than HMAC but requires a fresh nonce per message.

OpenSSL Commands — Hashing

```
# Compute SHA-256 of a file
openssl dgst -sha256 myfile.pdf

# Compute SHA-512
openssl dgst -sha512 myfile.pdf

# HMAC-SHA-256 with a key
openssl dgst -sha256 -hmac 'mysecretkey' myfile.pdf

# Hash a string (pipe input)
echo -n 'Hello World' | openssl dgst -sha256

# List all available digest algorithms
openssl list -digest-algorithms
```

Ch. 3

Symmetric Encryption

AES, ChaCha20, modes of operation

How Symmetric Encryption Works

🧒 **[ELI5] ELI5 — Explain Like I'm Five**

You and your friend share one key — like a physical key to the same padlock. You lock the box with it; your friend unlocks it. The problem: how do you give them the key without someone stealing it on the way? That is why asymmetric encryption was invented — but once you have a shared key, symmetric encryption is very fast.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**

Symmetric encryption uses the SAME key to encrypt and to decrypt. It is very fast — suitable for encrypting large files or streams. AES (Advanced Encryption Standard) is the global standard, used in Wi-Fi, HTTPS, disk encryption, and almost everything else.
The main challenge is key distribution: how do both parties get the same key securely? Answer: use asymmetric encryption or a key agreement protocol to exchange it.

⚙️ **Intermediate — Some Technical Background**

AES is a block cipher — it operates on fixed 128-bit (16-byte) blocks. The key can be 128, 192, or 256 bits. The cipher is iterated through 10/12/14 rounds (for 128/192/256-bit keys) of SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations. Raw block cipher (ECB mode) is insecure for most uses — a mode of operation is required to handle messages longer than one block and to add non-determinism via an IV or nonce.

🔬 **[EXPERT] Expert — Deep Technical Detail**

AES is an SP-network (Substitution-Permutation Network) instantiating the Rijndael cipher selected in the NIST AES competition (2001). Security proofs use the ideal cipher model. AES-NI hardware instructions (Intel Westmere+, AMD Bulldozer+, ARM Cortex-A53+) provide ~1 cycle/byte throughput, removing the performance advantage once held by ChaCha20. ChaCha20 (Bernstein, 2008) is an ARX (Add-Rotate-XOR) stream cipher — preferred on platforms without AES-NI. ChaCha20-Poly1305 is the IETF standard AEAD (RFC 8439). AES-GCM uses GHASH (a polynomial MAC over GF(2^128)) — nonce reuse catastrophically breaks confidentiality AND integrity; a single repeated nonce leaks the authentication key.

Modes of Operation

Mode	Provides Auth?	Parallelisable ?	IV reuse risk	Use case
ECB	No	Yes (enc+dec)	CATASTROPHIC — never use	Never — exposes patterns
CBC	No	Dec only	High — IV must be random	Legacy TLS, disk encryption
CTR	No	Yes	CATASTROPHIC if repeated	Streaming data with separate MAC
GCM	Yes (AEAD)	Yes	CATASTROPHI	TLS 1.3, SSH, preferred default

			C — see below	
CCM	Yes (AEAD)	No	High	Embedded/IoT constrained environments
SIV	Yes (AEAD)	Partial	Safe — nonce-misuse resistant	Deterministic encryption
ChaCha20-Poly1305	Yes (AEAD)	Partial	CATASTROPHIC	Mobile, no AES-NI hardware

AES-GCM in Depth — The Recommended Choice

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
AES-GCM is the recommended mode. It encrypts your data AND produces an authentication tag that detects any tampering. The tag is like a seal — if even one byte changes, the seal breaks. This combined encrypt-then-authenticate is called AEAD (Authenticated Encryption with Associated Data).

⚙️ **Intermediate — Some Technical Background**
GCM requires a unique 96-bit nonce per message (same key). The nonce does NOT need to be secret — it is sent alongside the ciphertext. The authentication tag is typically 128 bits. Associated Data (AD) can be included — it is authenticated but not encrypted (useful for packet headers, file metadata). Decryption MUST check the tag before releasing plaintext.

🔬 **[EXPERT] Expert — Deep Technical Detail**
GCM = CTR mode encryption + GHASH authentication. GHASH is H(A || C || len(A) || len(C)) evaluated as a polynomial over GF(2^128) with the hash subkey H = E_K(0^128).
Nonce reuse leaks H: given two ciphertexts (C1,T1) and (C2,T2) with the same nonce, T1 XOR T2 = GHASH(A1||C1) XOR GHASH(A2||C2), which with known C1,C2 allows H recovery and full tag forgery. Counter wrapping at 2^32 is a secondary limit (~64 GiB per key/nonce).
AES-GCM-SIV (RFC 8452) is nonce-misuse resistant: nonce reuse reveals only that the same plaintext was encrypted twice — it does NOT leak the auth key.

OpenSSL Commands — Symmetric Encryption

```
# Encrypt a file with AES-256-GCM (passphrase-derived key)
openssl enc -aes-256-gcm -salt -pbkdf2 -iter 600000 \
    -in plaintext.txt -out encrypted.bin -pass pass:'MyPassphrase'

# Decrypt
openssl enc -d -aes-256-gcm -pbkdf2 -iter 600000 \
    -in encrypted.bin -out decrypted.txt -pass pass:'MyPassphrase'

# Encrypt with raw hex key and IV (expert use)
openssl enc -aes-256-cbc -K <hex_key_64chars> -iv <hex_iv_32chars> \
    -in plain.txt -out cipher.bin

# List all cipher algorithms available
openssl list -cipher-algorithms
```

Ch. 4

Asymmetric / Public-Key Cryptography

RSA, ECDSA, Ed25519 — and how key pairs work

The Key-Pair Concept

🧒 **[ELI5] ELI5 — Explain Like I'm Five**

Imagine a special padlock you can give to anyone. They can lock a box with it — but only you have the key to open it. That padlock is your public key. The opening key is your private key. Never give away the opening key!

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**

A key pair has two mathematically linked halves. The public key can be shared freely. The private key must be kept secret. Anything encrypted with the public key can only be decrypted by the private key. Anything signed with the private key can be verified by anyone who has the public key. This solves the key distribution problem of symmetric crypto.

⚙️ **Intermediate — Some Technical Background**

RSA security relies on the difficulty of factoring large integers ($n = p \times q$, where p and q are large primes). ECDSA and Ed25519 rely on the elliptic curve discrete logarithm problem (ECDLP). For equivalent security, elliptic curve keys are much shorter: a 256-bit EC key ≈ 3072-bit RSA key. Key generation is random — the private key is a random scalar; the public key is derived by scalar multiplication on the curve.

🔬 **[EXPERT] Expert — Deep Technical Detail**

RSA encryption (OAEP): ciphertext = $(m^e) \bmod n$, where e is the public exponent (typically 65537) and n is the modulus. Decryption uses $d = e^{-1} \bmod \lambda(n)$ (Carmichael's function). RSA PKCS#1 v1.5 padding is vulnerable to Bleichenbacher's 1998 oracle attack — always use OAEP. ECDSA signatures are (r,s) pairs where $r = (kG).x \bmod n$, $s = k^{-1}(H(m)+r \cdot d) \bmod n$. Nonce k must be unique and secret per signature — the Sony PS3 vulnerability reused k, allowing private key recovery. Ed25519 (EdDSA on Curve25519) deterministically derives k from a hash of the private key and message, eliminating this hazard. Ed25519 also provides cofactor-free verification and resistance to fault attacks.

Algorithm Selection Guide

Algorithm	Key Size	Security Level	Speed	Recommended for
RSA-2048	2048-bit	~112-bit	Slow keygen	Legacy compatibility only
RSA-3072	3072-bit	~128-bit	Slow	Minimum for new RSA (NIST through 2031)
RSA-4096	4096-bit	~140-bit	Slow	High-security RSA, CA keys
ECDSA P-256	256-bit	~128-bit	Fast	TLS, code signing, FIPS environments
ECDSA P-384	384-bit	~192-bit	Medium	High-security TLS, government
Ed25519	256-bit	~128-bit	Very fast	SSH keys, JWT, modern signing

Ed448	448-bit	~224-bit	Medium	Extra security margin, rare use
X25519	256-bit	~128-bit	Very fast	Key agreement (DH), not signing

OpenSSL Commands — Key Generation & Management

```
# Generate Ed25519 key pair (recommended for new projects)
openssl genpkey -algorithm ed25519 -out private.pem
openssl pkey -in private.pem -pubout -out public.pem

# Generate ECDSA P-256 key pair
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out ec_priv.pem
openssl pkey -in ec_priv.pem -pubout -out ec_pub.pem

# Generate RSA 4096 key pair with AES-256-CBC passphrase protection
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 \
    -aes-256-cbc -out rsa_priv.pem
openssl pkey -in rsa_priv.pem -pubout -out rsa_pub.pem

# Inspect a private key
openssl pkey -in private.pem -text -noout

# Convert PEM to DER
openssl pkey -in private.pem -out private.der -outform DER

# Remove passphrase from a key
openssl pkey -in protected.pem -out unprotected.pem
```

Ch. 5

Entropy, Randomness & Key Derivation

CSPRNG, PBKDF2, Argon2id, HKDF

Why Randomness Is Foundational

🧒 **[ELI5] ELI5 — Explain Like I'm Five**

If a magician always picks the same card, you could guess his trick. Encryption keys need to be completely unpredictable — like rolling dice that are truly fair. Computers use special hardware noise (keystrokes, disk timing, CPU temperature) to create real randomness.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**

A Cryptographically Secure Pseudo-Random Number Generator (CSPRNG) produces numbers that are indistinguishable from true random. The OS provides this: /dev/urandom on Linux, CryptGenRandom on Windows, SecRandomCopyBytes on macOS. OpenSSL uses the OS CSPRNG.
Never use rand(), Math.random(), or Python's random module for cryptographic keys.

⚙️ **Intermediate — Some Technical Background**

OpenSSL maintains its own internal DRBG (Deterministic Random Bit Generator) seeded from the OS entropy pool. On Linux, it reads from getrandom() syscall (kernel 3.17+) or /dev/urandom. The DRBG is based on CTR-DRBG (NIST SP 800-90A) using AES-256.
Entropy starvation is rare on modern hardware (RDRAND/RDSEED instructions on Intel/AMD) but can occur in virtualised environments or early-boot firmware — significant for embedded systems like RP2350 where hardware RNG must be explicitly initialised.

🔬 **[EXPERT] Expert — Deep Technical Detail**

OpenSSL 3.x uses a provider-based DRBG chain: primary DRBG (seeded from OS) feeds per-thread DRBGs for performance. The DRBG supports prediction resistance mode (reseeds from entropy before each request). For FIPS 140-3 compliance, the DRBG must pass continuous health tests and the approved DRBG mechanism must be used.
Key derivation: HKDF (RFC 5869) = Extract(salt, IKM) || Expand(PRK, info, L).
Extract produces a pseudorandom key PRK from potentially non-uniform IKM.
Expand derives output keying material of arbitrary length. Argon2id parameters (m=65536 KiB, t=3, p=4) are the current OWASP-recommended minimum for password hashing.

OpenSSL Commands — Random & Key Derivation

```
# Generate 32 random bytes as hex (use for AES-256 key)
openssl rand -hex 32

# Generate 16 random bytes as Base64
openssl rand -base64 16

# Derive a key from a password using PBKDF2-SHA256 (600,000 iterations)
openssl kdf -keylen 32 -kdfopt digest:SHA256 \
    -kdfopt pass:MyPassword -kdfopt salt:$(openssl rand -hex 16) \
    -kdfopt iter:600000 PBKDF2

# Generate Diffie-Hellman parameters (for legacy TLS configurations)
openssl dhparam -out dhparam.pem 4096
```

```
# Generate a secure random prime
openssl prime -generate -bits 512
```

Ch. 6

PKI & X.509 Certificates

Certificate authorities, chains of trust, CSRs, CRLs, OCSP

What Is a Certificate?

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
Your passport is issued by a government that everyone trusts. It has your photo and name on it, and a stamp proving it is real. A digital certificate is the same idea — it has a website's public key and a stamp (signature) from a trusted authority proving it belongs to that website.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
An X.509 certificate is a digital document that binds a public key to an identity (a person, company, server, or device). It is signed by a Certificate Authority (CA). When your browser connects to https://example.com, the server presents its certificate. Your browser checks that a trusted CA signed it, and that the domain name matches. This is how HTTPS works.

⚙️ **Intermediate — Some Technical Background**
An X.509 v3 certificate contains: Subject (DN — Distinguished Name), Issuer DN, Serial Number, Validity period (NotBefore / NotAfter), Subject Public Key Info (algorithm + public key), and v3 Extensions: Subject Alternative Names (SANs), Key Usage, Extended Key Usage (EKU), Basic Constraints (is this a CA?), CRL Distribution Points, Authority Information Access (OCSP URL). The certificate is ASN.1 DER-encoded and signed by the issuer's private key over the TBSCertificate structure.

🔬 **[EXPERT] Expert — Deep Technical Detail**
RFC 5280 defines the X.509 v3 profile. Certificate validation (RFC 5280 §6) involves: chain building (from EE to trust anchor), signature verification at each link, validity period check, policy processing (CPS / CP OIDs), name constraints, revocation checking (CRL or OCSP). The pathLenConstraint in Basic Constraints limits intermediate CA depth. Name constraints allow a CA to restrict issuance to specific DNS/IP subtrees — critical for technically-constrained sub-CAs in multi-tenant environments. Certificate Transparency (RFC 9162) requires CAs to log all issued certs to public append-only logs; browsers now require SCTs (Signed Certificate Timestamps) embedded in or stapled to certificates.

Certificate Authority Hierarchy

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
A CA hierarchy is like a chain of trust. The Root CA is at the top — it is pre-installed in your OS or browser. It signs Intermediate CAs. Intermediate CAs sign end-entity certificates (for websites, users, devices). The root is kept offline for safety.

⚙️ **Intermediate — Some Technical Background**
Typical hierarchy: Root CA (offline, 20-year validity, 4096-bit RSA or P-384) →
Intermediate CA (online, 10-year validity, issues end-entity certs) →
End-entity cert (1–2 year validity, includes SANs).
The intermediate is online so the root's private key can remain air-gapped.
If the intermediate is compromised, it can be revoked without replacing the root.

🔬 **[EXPERT] Expert — Deep Technical Detail**
Root programs (Mozilla, Apple, Microsoft, Google) define requirements for publicly-trusted CAs via the CA/Browser Forum Baseline Requirements (BRs) and Root Store policies. BRs mandate: max 398-day validity for TLS certs (Apple requirement, 2020), CT logging, CAA record checking (DNS CAA, RFC 8659), and prohibited algorithms (SHA-1 since 2017). Cross-certification and bridge CAs allow interoperability between independent PKI domains. RPKI (Resource PKI, RFC 6487) applies X.509 to BGP route origin validation, using resource extensions to bind IP prefixes and ASNs to certificates.

CSR — Certificate Signing Request

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
A CSR is a form you fill in and send to a CA asking for a certificate. It contains your public key and your identity details (domain name, organisation, country). The CA verifies the information, then issues a signed certificate.

⚙️ **Intermediate — Some Technical Background**
A CSR (PKCS#10, RFC 2986) is ASN.1-encoded and contains: Subject DN, Subject Public Key Info, and optionally requested extensions (SANs, key usage). It is self-signed with the applicant's private key — proving key possession. The CA may ignore or override requested extensions depending on their policy.

OpenSSL Commands — PKI Operations

```
# Generate a private key and CSR in one step
openssl req -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes \
    -keyout server.key -out server.csr \
    -subj '/CN=example.com/O=My Company/C=IN'

# Add SANs via config file (san.cnf):
# [req] req_extensions = v3_req
# [v3_req] subjectAltName = DNS:example.com,DNS:www.example.com,IP:1.2.3.4
openssl req -new -key server.key -out server.csr -config san.cnf

# Create a self-signed certificate (dev/testing)
openssl req -x509 -newkey ed25519 -nodes -days 365 \
    -keyout selfsigned.key -out selfsigned.crt \
    -subj '/CN=localhost'

# Create a Root CA
openssl req -x509 -newkey rsa:4096 -days 7300 -nodes \
    -keyout rootCA.key -out rootCA.crt \
    -subj '/CN=My Root CA/O=My Org/C=IN' \
    -addext 'basicConstraints=critical,CA:TRUE'

# Sign a CSR with the root CA
openssl x509 -req -in server.csr -CA rootCA.crt -CAkey rootCA.key \
    -CAcreateserial -days 397 -sha256 -out server.crt \
    -extfile san.cnf -extensions v3_req

# Inspect a certificate
openssl x509 -in server.crt -text -noout

# Verify a certificate chain
openssl verify -CAfile rootCA.crt server.crt
```

```
# Check TLS certificate of a remote host
openssl s_client -connect example.com:443 -servername example.com </dev/null | \
    openssl x509 -text -noout

# Create PKCS#12 bundle (for web server deployment)
openssl pkcs12 -export -out bundle.pfx \
    -inkey server.key -in server.crt -certfile rootCA.crt

# Generate CRL
openssl ca -gencrl -keyfile rootCA.key -cert rootCA.crt \
    -out rootCA.crl -crldays 30

# Revoke a certificate
openssl ca -revoke server.crt -keyfile rootCA.key -cert rootCA.crt
```

Ch. 7

Digital Signatures & Signing Formats

PKCS#7, CAdES, PAdES, RFC 3161 timestamps

What Is a Digital Signature?

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
When you sign a paper document with your pen, it proves you agreed to it. A digital signature does the same — it uses your private key to produce a unique stamp on a file. Anyone with your public key can check the stamp is genuine and the file has not changed.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
A digital signature is created by hashing a file and encrypting the hash with your private key. To verify: the verifier hashes the file themselves, decrypts your signature with your public key, and checks the two hashes match. If the file was changed even slightly, the hashes will not match and verification fails.

⚙️ **Intermediate — Some Technical Background**
Signing = Sign(privKey, H(message)). Verification = Verify(pubKey, signature, H(message)). For RSA: signature = H(m)^d mod n. For ECDSA: (r,s) pair computed from random nonce k. For Ed25519: deterministic (no random nonce). The signature covers the hash — so the hash algorithm is part of the signature scheme (e.g., SHA256withRSA, SHA256withECDSA). Signing formats: raw signature bytes (primitive), PKCS#7/CMS (structured, with cert), CAdES (ETSI advanced electronic signature), PAdES (PDF signatures), XAdES (XML).

🔬 **[EXPERT] Expert — Deep Technical Detail**
CMS (RFC 5652) SignedData structure embeds: version, DigestAlgorithms, encapContentInfo (optional embedded content — omit for detached), certificates, crls, and SignerInfos. Each SignerInfo contains: version, signerIdentifier (issuerAndSerialNumber or subjectKeyIdentifier), digestAlgorithm, signedAttrs (required: contentType + messageDigest; optional: signingTime, signingCertificateV2), signatureAlgorithm, signature, unsignedAttrs. CAdES-B (ETSI EN 319 122) adds mandatory SigningCertificateV2 signed attribute and precise algorithm identifiers. CAdES-T adds an RFC 3161 timestamp over the signature. CAdES-LT adds full revocation data. CAdES-LTA adds archive timestamps for long-term validity.

RFC 3161 — Trusted Timestamps

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
A timestamp token is proof from a trusted third party (Time Stamping Authority, TSA) that a particular piece of data existed at a specific time. This is important for legal documents: even after a signing certificate expires, you can prove the signature was made while the certificate was valid.

🔬 **[EXPERT] Expert — Deep Technical Detail**
RFC 3161 TSP (Time-Stamp Protocol): client sends a TimeStampReq containing a hash of the data and optionally a requested policy OID. The TSA responds with a TimeStampResp containing a TSTInfo (RFC 3161 §2.4.2) CMS SignedData. TSTInfo includes: policy OID, messageImprint (hash algorithm + hash value), serialNumber, genTime (UTC, up to millisecond precision), and optionally TSA name. The TSA's signing certificate must have id-kp-timeStamping EKU. Free public TSAs: Freetsa.org, Digicert TSA, Sectigo TSA.

OpenSSL Commands — Signing & Verification

```
# Sign a file (detached signature)
openssl dgst -sha256 -sign private.pem -out file.sig file.bin

# Verify a detached signature
openssl dgst -sha256 -verify public.pem -signature file.sig file.bin

# Create a PKCS#7 / CMS detached signature (includes signer certificate)
openssl cms -sign -in document.pdf -signer cert.pem -inkey private.pem \
    -out document.p7s -outform DER -binary -nodetach

# Verify a CMS signature
openssl cms -verify -in document.p7s -inform DER -binary \
    -CAfile rootCA.crt -out verified_content.pdf

# Request an RFC 3161 timestamp from Freetsa.org
openssl ts -query -data document.pdf -sha256 -no_nonce -out request.tsq
curl -s -H 'Content-Type: application/timestamp-query' \
    --data-binary @request.tsq https://freetsa.org/tsr -o timestamp.tsr

# Verify the timestamp response
openssl ts -verify -in timestamp.tsr -data document.pdf \
    -CAfile freetsa_ca.pem
```

Ch. 8

S/MIME & Email Encryption

Encrypting and signing email end-to-end

How S/MIME Works

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
S/MIME is like sealing a letter in a locked envelope before posting. The recipient has the only key to open it. You can also stamp the envelope with wax using your signet ring — the recipient checks the stamp to know it came from you.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
S/MIME (Secure/Multipurpose Internet Mail Extensions) adds encryption and digital signatures to email. To send an encrypted email to someone, you need their certificate (containing their public key). To sign your email, you use your own private key. Recipients verify your signature using your certificate, which you attach to signed emails.

⚙️ **Intermediate — Some Technical Background**
S/MIME encryption uses a hybrid scheme: a random session key (e.g., AES-256) encrypts the message body. The session key is then encrypted with each recipient's public key (RSA-OAEP or ECDH key agreement) and included in the CMS EnvelopedData structure. This allows one encrypted message to be addressed to multiple recipients, each with their own copy of the session key encrypted to their public key.
S/MIME signing uses CMS SignedData — the same structure used for document signing.

🔬 **[EXPERT] Expert — Deep Technical Detail**
CMS EnvelopedData (RFC 5652 §6): encryptedContentInfo uses the session key; recipientInfos contains one entry per recipient — either KeyTransRecipientInfo (RSA key transport) or KeyAgreeRecipientInfo (ECDH key agreement per RFC 5753, using static-ephemeral ECDH with key derivation via ANSI-X9.63-KDF or HKDF). S/MIME v4 (RFC 8551) mandates AES-128-CBC minimum, recommends AES-256-CBC, and allows AES-GCM as an AuthEnvelopedData structure. Key management challenge: obtaining and validating recipients' certificates. Directory services (LDAP), WKD (Web Key Directory, draft-koch-openpgp-webkey-service), and S/MIME certificate discovery via DNS (SMIMEA, RFC 8162) all attempt to solve this.

S/MIME vs PGP/GPG — A Comparison

Property	S/MIME	PGP/OpenPGP
Trust model	Hierarchical CA — third party issues certs	Web of Trust — users sign each other's keys
Key format	X.509 certificate	OpenPGP keyring (.asc, .gpg)
Email clients	Outlook, Thunderbird, Apple Mail — native	Thunderbird (Enigmail/built-in), K-9 Mail
Cert cost	Free (self-signed) or paid (public CA)	Free — anyone can generate a PGP key
Key discovery	Certificate attached to signed email;	Keyservers (keys.openpgp.org), WKD

	LDAP	
Encryption alg	AES-128/256 (CMS)	AES-256, Camellia (OpenPGP)
Standardised by	IETF RFC 8551	IETF RFC 4880 (OpenPGP), RFC 9580 (v6)
Best for	Corporate email, regulated industries	Individuals, journalists, activists

OpenSSL Commands — S/MIME

```
# Generate a self-signed S/MIME certificate
openssl req -x509 -newkey rsa:4096 -nodes -days 365 \
    -keyout smime.key -out smime.crt \
    -subj '/CN=user@example.com/emailAddress=user@example.com' \
    -addext 'keyUsage=digitalSignature,keyEncipherment' \
    -addext 'extendedKeyUsage=emailProtection'

# Sign an email message
openssl smime -sign -in message.txt -signer smime.crt \
    -inkey smime.key -out signed.eml

# Verify a signed email
openssl smime -verify -in signed.eml -CAfile rootCA.crt

# Encrypt an email for a recipient
openssl smime -encrypt -aes256 -in message.txt \
    -out encrypted.eml recipient_cert.pem

# Decrypt a received encrypted email
openssl smime -decrypt -in encrypted.eml \
    -inkey smime.key -recip smime.crt -out plaintext.txt

# Export key+cert as PKCS#12 for Thunderbird/Outlook import
openssl pkcs12 -export -out smime_bundle.p12 \
    -inkey smime.key -in smime.crt
```

Ch. 9

TLS & HTTPS Internals

How the padlock in your browser actually works

TLS From First Principles

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
When you see the padlock in your browser, it means your computer and the website did a secret handshake first. They agreed on a code word (session key) by passing locked boxes back and forth that nobody else could open. Now all your messages to the website are scrambled with that code word.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
TLS (Transport Layer Security) is the protocol behind HTTPS. It provides three things:
1. Authentication — you are really talking to example.com (via certificates).
2. Confidentiality — your data is encrypted so nobody on the network can read it.
3. Integrity — nobody can tamper with the data in transit without detection.
TLS 1.3 is the current standard (2018). TLS 1.0 and 1.1 are deprecated.

⚙️ **Intermediate — Some Technical Background**
TLS 1.3 handshake (RFC 8446): client sends ClientHello with supported cipher suites and key shares (X25519, P-256). Server responds with ServerHello (chosen suite + key share), Certificate, CertificateVerify (signature over handshake transcript), and Finished (HMAC). Client verifies the certificate chain and signature, sends Finished. A shared secret is derived via ECDH key agreement, then expanded via HKDF into four traffic keys (client/server handshake and application data). 0-RTT data is optionally sent with the first flight — but is replay-vulnerable and should only be used for idempotent requests.

🔬 **[EXPERT] Expert — Deep Technical Detail**
TLS 1.3 key schedule: HKDF-Extract(0, 0) → early_secret; HKDF-Extract(DHE, early_secret) → handshake_secret; HKDF-Extract(0, handshake_secret) → master_secret. Derived secrets: client_handshake_traffic_secret, server_handshake_traffic_secret, exporter_master_secret, resumption_master_secret. Per-record keys derived via HKDF-Expand-Label.
Mandatory cipher suites: TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256. Key exchange groups: X25519 (preferred), P-256, P-384, X448, FFDHE2048-8192. TLS 1.3 removes RSA key exchange (non-forward-secret), static DH, CBC mode ciphers, RC4, 3DES, MD5/SHA-1, and compression.
Session resumption uses PSK (Pre-Shared Key) with NewSessionTicket. Encrypted SNI (ESNI / ECH, RFC draft) hides the server name from passive observers by encrypting ClientHello.

TLS Version Comparison

Feature	TLS 1.0/1.1	TLS 1.2	TLS 1.3
Status	DEPRECATED (RFC 8996)	Supported (configure carefully)	Current standard
Key exchange	RSA, DHE, ECDHE	RSA, DHE, ECDHE	ECDHE, DHE only (no RSA)

Forward secrecy	Optional	Optional	Mandatory
Handshake RTTs	2 RTT	2 RTT	1 RTT (0-RTT optional)
Cipher suites	Many (incl. RC4, 3DES)	Many (incl. CBC)	3 AEAD-only suites
Certificate auth	RSA, DSA	RSA, ECDSA	RSA, ECDSA, Ed25519
Padding oracle	Vulnerable (POODLE)	Mitigated	Not applicable
Known attacks	BEAST, POODLE, CRIME	Possible with weak config	None known

OpenSSL Commands — TLS Inspection

```
# Full TLS handshake inspection of a remote server
openssl s_client -connect example.com:443 -servername example.com \
    -tls1_3 -status -tlsextdebug

# Show only the certificate chain
openssl s_client -connect example.com:443 -showcerts 2>/dev/null

# Check certificate expiry date
echo | openssl s_client -connect example.com:443 2>/dev/null | \
    openssl x509 -noout -dates

# Test which TLS versions a server supports
for v in ssl3 tls1 tls1_1 tls1_2 tls1_3; do
  echo -n "$v: "; openssl s_client -connect host:443 -$v 2>&1 | grep -E 'Cipher|
alert'
done

# Run a test TLS server with a self-signed certificate
openssl s_server -key server.key -cert server.crt -accept 4433 -www

# Test against the local server
openssl s_client -connect localhost:4433

# Show supported cipher suites for TLS 1.3
openssl ciphers -v 'TLSv1.3'

# Generate a strong TLS configuration snippet (DH params)
openssl dhparam -out /etc/ssl/dhparam.pem 4096
```

Recommended TLS Configuration (Nginx Example)

```
# Nginx — TLS 1.3 + 1.2, strong ciphers (Mozilla Intermediate profile)
ssl_protocols              TLSv1.2 TLSv1.3;
ssl_ciphers                ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-
SHA256:
                           ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:
                           ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-
POLY1305;
ssl_prefer_server_ciphers  off;   # Let client choose (TLS 1.3 doesn't use this)
ssl_session_timeout        1d;
ssl_session_cache          shared:SSL:10m;
```

```
ssl_stapling                on;    # OCSP stapling
ssl_stapling_verify         on;
add_header Strict-Transport-Security 'max-age=63072000' always;
```

Ch. 10

OpenSSL CLI — Command Reference

Every major subcommand explained with examples

OpenSSL Command Structure

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
OpenSSL is a single executable with many subcommands: openssl <command> [options].
Type 'openssl help' to list all commands. Type 'openssl <command> -help' for options.
Most commands read and write PEM files (Base64 text) or DER files (binary).
PEM files start with -----BEGIN ...-----. DER is the binary equivalent.

🔬 **[EXPERT] Expert — Deep Technical Detail**
OpenSSL 3.x introduced the provider architecture: FIPS, legacy, and default providers are loadable modules. Algorithm availability depends on which providers are loaded. The openssl.cnf file (usually /etc/ssl/openssl.cnf or /usr/lib/ssl/openssl.cnf) controls defaults. Environment variable OPENSSL_CONF overrides the config path. The ENGINE subsystem (deprecated in 3.x in favour of providers) allowed hardware acceleration.

Core Subcommand Reference

openssl genpkey	Generate private keys (RSA, EC, Ed25519, X25519, DH). Preferred over legacy genrsa/ecparam.
openssl pkey	Read, write, convert, and inspect private/public keys. Replaces rsa/ec for most tasks.
openssl req	Create CSRs and self-signed certificates. Reads subject DN and extensions from config or -subj.
openssl x509	Display, convert, and sign X.509 certificates. Use -text -noout to inspect.
openssl ca	Simple CA — sign CSRs, maintain a serial/index database, generate CRLs.
openssl verify	Verify a certificate chain against a trust store.
openssl pkcs12	Create/extract PKCS#12 (.pfx/.p12) bundles containing key + cert + chain.
openssl pkcs7	View and convert PKCS#7 / CMS structures. Use openssl cms for signing/verifying.
openssl cms	CMS sign, verify, encrypt, decrypt (S/MIME and document signing).
openssl smime	Legacy S/MIME interface (wraps cms). Use for email operations.
openssl dgst	Hash files, compute HMACs, sign digests, verify signatures.
openssl enc	Symmetric encryption/decryption. Supports AES, ChaCha20, 3DES, and others.
openssl rand	Generate cryptographically secure random bytes in hex, Base64, or binary.
openssl s_client	TLS client — connect to a server, inspect certificates, test handshake.

openssl s_server	TLS server — for testing. Accepts connections on a specified port.
openssl speed	Benchmark algorithm performance on your hardware.
openssl ts	RFC 3161 timestamp request and response handling.
openssl ocsp	Send OCSP requests and parse responses for revocation status.
openssl crl	Inspect and convert Certificate Revocation Lists.
openssl dhparam	Generate Diffie-Hellman parameters for TLS/OpenVPN.
openssl prime	Test primality or generate random primes.
openssl asn1parse	Parse and display ASN.1 DER or PEM-encoded data structures.
openssl list	List available algorithms, providers, ciphers, digests, KDFs, etc.

Essential One-Liners — Quick Reference

```
### KEYS ---------------------------------------------------------------
openssl genpkey -algorithm ed25519 -out key.pem                          # Ed25519
key
openssl genpkey -algorithm rsa -pkeyopt rsa_keygen_bits:4096 -out rsa.pem # RSA
4096
openssl pkey -in key.pem -text -noout                                     # Inspect
key
openssl pkey -in key.pem -pubout -out pub.pem                             # Extract
public

### CERTS --------------------------------------------------------------
openssl req -x509 -newkey ed25519 -nodes -days 365 \
    -keyout k.pem -out c.pem -subj '/CN=localhost'                         # Self-
signed
openssl x509 -in cert.pem -text -noout                                    # Inspect
cert
openssl x509 -in cert.pem -noout -dates                                   # Expiry
dates
openssl x509 -in cert.pem -noout -fingerprint -sha256                     # SHA-256
fingerprint

### HASHING ------------------------------------------------------------
openssl dgst -sha256 file                                                  # Hash a
file
openssl dgst -sha256 -hmac 'key' file                                     # HMAC
echo -n 'text' | openssl dgst -sha256                                     # Hash a
string

### ENCRYPTION ---------------------------------------------------------
openssl enc -aes-256-gcm -salt -pbkdf2 -in f.txt -out f.enc               # Encrypt
openssl enc -d -aes-256-gcm -pbkdf2 -in f.enc -out f.txt                  # Decrypt
openssl rand -hex 32                                                       # Random
256-bit key

### TLS ----------------------------------------------------------------
openssl s_client -connect host:443 -servername host </dev/null            # Check
TLS cert
echo | openssl s_client -connect host:443 2>/dev/null | \
    openssl x509 -noout -dates                                            # Expiry
check
openssl s_server -key k.pem -cert c.pem -accept 4433 -www                # Test
server
```

```
### FORMAT CONVERSION ----------------------------------------------------
openssl x509 -in cert.pem -out cert.der -outform DER                    # PEM → DER
openssl x509 -in cert.der -inform DER -out cert.pem                     # DER → PEM
openssl pkcs12 -export -inkey k.pem -in c.pem -out b.p12               # Make
PKCS#12
openssl pkcs12 -in b.p12 -nodes -out combined.pem                       # Unpack
PKCS#12

### DIAGNOSTICS ----------------------------------------------------------
openssl version -a                                                        # Full
version info
openssl list -providers                                                   # Loaded
providers
openssl list -cipher-algorithms                                           # All
ciphers
openssl speed rsa4096 aes-256-gcm ed25519                               # Benchmark
```

Ch. 11

Post-Quantum Cryptography

ML-KEM, ML-DSA, SLH-DSA — preparing for the quantum era

The Quantum Threat Explained

🧒 **[ELI5] ELI5 — Explain Like I'm Five**

Today's locks (RSA, ECDSA) are based on maths problems that normal computers cannot solve quickly. A quantum computer — which does not widely exist yet, but is being built — could solve those specific problems very fast and break those locks. Scientists are building new kinds of locks that even quantum computers cannot break.

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**

Quantum computers use quantum mechanics to explore many possible solutions simultaneously. Shor's algorithm (1994) can break RSA and ECDSA efficiently on a sufficiently large quantum computer by solving integer factorisation and discrete logarithm in polynomial time. Grover's algorithm halves the effective key length of symmetric ciphers — so AES-256 becomes equivalent to AES-128 against a quantum adversary. Solution: double key lengths for symmetric (already done with AES-256), and replace RSA/ECDSA with new algorithms.

⚙️ **Intermediate — Some Technical Background**

The timeline is uncertain: cryptographically relevant quantum computers (CRQCs) require millions of fault-tolerant logical qubits; the current state of the art is a few thousand noisy physical qubits (2024). NIST estimates a significant probability of CRQCs within 15–20 years. However, 'harvest now, decrypt later' attacks are already a threat — adversaries can record encrypted traffic today and decrypt it when CRQCs become available. This makes migration urgency proportional to the sensitivity lifetime of the data.

🔬 **[EXPERT] Expert — Deep Technical Detail**

NIST PQC standardisation (2024 final standards): ML-KEM (FIPS 203, lattice-based KEM, formerly Kyber), ML-DSA (FIPS 204, lattice-based signature, formerly Dilithium), SLH-DSA (FIPS 205, stateless hash-based signature, formerly SPHINCS+).
FALCON (FN-DSA) is under separate standardisation (FIPS 206 draft) for compact signatures.
Security foundations: ML-KEM/ML-DSA rely on the Module Learning With Errors (MLWE) and Module Short Integer Solution (MSIS) hardness assumptions — conjectured quantum-resistant.
SLH-DSA relies only on hash function security (PRF, collision resistance) — the most conservative choice. Hybrid schemes (classical + PQC) are recommended during transition:
X25519+ML-KEM-768 for key agreement (IETF draft-ietf-tls-hybrid-design),
Ed25519+ML-DSA-65 for signing.

NIST Post-Quantum Standards — At a Glance

Standard	Algorithm	Type	Key sizes	Notes
FIPS 203	ML-KEM-512/768/1024	Key encapsulation	800B–1568B public key	Replaces ECDH/RSA key exchange. Kyber finalist.
FIPS 204	ML-DSA-44/65/87	Digital signature	1312B–2592B public key	Replaces ECDSA/Ed25519. Fast verify. Dilithium finalist.
FIPS 205	SLH-DSA	Digital	32–64B public key	Hash-based. Large signatures

	(SPHINCS+)	signature		(8–50 KB). Conservative.
FIPS 206	FN-DSA (FALCON)	Digital signature	897B–1793B public key	Compact sigs (~690B). Complex implementation (NTRU lattice).

OQS-OpenSSL — Experimenting Today

🌱 **[BEGINNER] Beginner — No Prior Knowledge Needed**
The Open Quantum Safe (OQS) project provides a modified version of OpenSSL that includes all the new post-quantum algorithms. You can install it alongside your regular OpenSSL to experiment with the new algorithms without breaking anything.

⚙️ **Intermediate — Some Technical Background**
OQS-Provider (github.com/open-quantum-safe/oqs-provider) is a loadable provider for OpenSSL 3.x that adds PQC algorithms including all NIST finalists and alternates.
It enables hybrid key exchange in TLS 1.3 (e.g., x25519_kyber768) and PQC certificates.
For testing only — OQS implementations are not yet FIPS-validated.

🔬 **[EXPERT] Expert — Deep Technical Detail**
OQS liboqs implements PQC primitives in C with optional AVX2/NEON acceleration.
The oqs-provider registers algorithm OIDs and integrates with OpenSSL's EVP layer.
Hybrid TLS groups (e.g., x25519_mlkem768) concatenate classical + PQC key material:
shared secret = HKDF(x25519_ss || mlkem_ss, 'hybrid label'). IETF draft-ietf-tls-hybrid-design defines the wire format. For certificate signing: hybrid X.509 approaches include alternative signatures (draft-ietf-lamps-pq-composite-sigs) where both classical and PQC signatures are embedded in a composite SignatureAlgorithm OID.

```
# Build OQS-Provider (requires cmake, liboqs)
git clone https://github.com/open-quantum-safe/oqs-provider
cd oqs-provider && mkdir build && cd build
cmake -DOPENSSL_ROOT_DIR=$(openssl version -d | cut -d'"' -f2)/.. ..
make && sudo make install

# List PQC algorithms available via OQS provider
openssl list -kem-algorithms -provider oqs
openssl list -signature-algorithms -provider oqs

# Generate an ML-DSA-65 (Dilithium3) key pair
openssl genpkey -provider oqs -algorithm mldsa65 -out pqc_private.pem
openssl pkey -provider oqs -in pqc_private.pem -pubout -out pqc_public.pem

# Create a self-signed certificate using ML-DSA-65
openssl req -provider oqs -x509 -new -key pqc_private.pem \
    -out pqc_cert.pem -days 365 -subj '/CN=PQC Test'

# TLS 1.3 server with hybrid key exchange (X25519 + ML-KEM-768)
openssl s_server -provider oqs -provider default \
    -key server.key -cert server.crt -accept 4433 \
    -groups x25519_mlkem768

# TLS 1.3 client using hybrid key exchange
openssl s_client -provider oqs -provider default \
    -connect localhost:4433 -groups x25519_mlkem768
```

Migration Roadmap Recommendations

Immediate (now)	Audit all long-lived data and keys. Identify systems using RSA/ECDSA. Ensure AES-256 (not AES-128) for sensitive data at rest.
Short term (1–2 years)	Deploy TLS 1.3 everywhere. Enable hybrid PQC key exchange in TLS (x25519_mlkem768). Test OQS-Provider in staging environments.
Medium term (3–5 years)	Issue new certificates using ML-DSA or FN-DSA. Migrate code signing to PQC. Standardise on FIPS 203/204/205 compliant libraries.
Long term (5–10 years)	Full deprecation of RSA and ECDSA for new systems. Archive verification of legacy signatures using hybrid or classical schemes for continuity.
sslOpenCrypt plan	v2.x will include PQC key generation, signing, and TLS inspection via OQS-Provider. Educational animations of lattice problems and sponge-based hash agility are planned for the Educational Hub.

Part III

India Digital Signature Ecosystem

Class 3 DSC tokens · Aadhaar eSign · DigiLocker · Government portals · sslOpenCrypt Module 10

Ch. 12

India's Digital Signature Ecosystem

Legal foundation, India PKI hierarchy, regulatory framework

12.1 Legal Foundation — IT Act 2000

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
India passed a law in 2000 saying a digital signature is as legal as your pen signature.
The government set up the CCA — a special office — to make sure only trusted organisations issue these signatures, using technology that cannot be faked.

🌱 **[BEGINNER] Beginner — No Prior Knowledge**
The Information Technology Act 2000 (IT Act) gives legal recognition to digital signatures.
Section 3 defines a valid digital signature: asymmetric cryptography + hash function.
Section 3A (2008 amendment) added Aadhaar eSign as a second legally valid method.
Digital signatures are admissible in court and equivalent to handwritten signatures.
Class 2 DSCs were discontinued on 01-Jan-2021. Only Class 3 is now issued.

⚙️ **[INTERMEDIATE] Intermediate — Technical Background**
CCA (Controller of Certifying Authorities) is appointed under Section 17 of the IT Act.
It licenses CAs, operates the RCAI root, and sets policy via CCA-CP and CCA-IVG.
July 2024 IVG update: mandatory video KYC for all new Class 3 DSC applications.
Algorithm requirement: RSA-2048 minimum (RSA-4096 for CA keys), SHA-256 signatures.
EC keys (P-256) permitted. Key storage: FIPS 140-2 Level 2+ certified hardware token.

🔬 **[EXPERT] Expert — Deep Technical Detail**
India PKI follows X.509 v3 (RFC 5280). CCA-CP defines per-assurance-level policies.
Class 3 certificates embed PAN number and optionally masked Aadhaar, DIN, or email in SANs.
Signature algorithms: SHA256withRSA or SHA256withECDSA. All CAs must operate OCSP.
RCAI is cross-certified with MeitY eSign framework HSM roots for eSign interoperability.
Certificate Transparency (SCTs) not yet mandated in India PKI — unlike CA/B Forum BRs.

12.2 India PKI Hierarchy

Level	Entity	Role	Key size / Validity
Root	RCAI — Root Certifying Authority of India (CCA)	Signs all licensed CA public keys. Offline HSM. Pre-installed on GoI portals.	RSA-4096 / 25 years
Intermediate	Licensed CAs: eMudhra, (n)Code, Capricorn, XtraTrust, SafeScrypt, CDAC, Protean, CDSL Ventures, Verasys, ~15 total	Issue Class 3 DSCs to end entities. Operate OCSP responders and publish CRLs.	RSA-4096 / 10–15 years
eSign	ESP sub-CAs (CDAC e-	Issue one-time signing certs for	RSA-2048 / 30 min (per-use)

sub-CA	Hastakshar, eMudhra, Protean, CDSL)	Aadhaar eSign. Keys on HSM, destroyed after use.	
End entity	Individuals, directors, organisations, devices	Hold Class 3 DSC on FIPS-certified USB token. Use for portal filings and document signing.	RSA-2048 or P-256 / 1–3 years

Ch. 13

Class 3 DSC USB Tokens

Hardware, drivers, PKCS#11, and OpenSSL integration

13.1 Token Hardware

🧒 **[ELI5] ELI5 — Explain Like I'm Five**
A DSC token is a small USB stick — like a very secure pen drive — that holds your digital signature. You plug it in, type a PIN, and the token signs documents for you. Nobody can copy your signature out of the token — it is built to refuse that.

🌱 **[BEGINNER] Beginner — No Prior Knowledge**
Common tokens in India: Feitian ePass2003 / HYP2003 (most common), SafeNet eToken 5110, WatchData Proxkey, MoserBaer tokens. All are FIPS 140-2 Level 2+ certified.
The token stores your RSA-2048 or P-256 private key in tamper-resistant hardware.
The key cannot be exported or copied. Signing happens inside the token.
You get the token from a licensed CA or their RA (Registration Authority).

⚙️ **[INTERMEDIATE] Intermediate — Technical Background**
Token interface: PKCS#11 (primary — all platforms) and Microsoft CAPI/CNG (Windows).
Vendor PKCS#11 libraries: libcastle.so.1.0.0 (ePass2003/HYP2003), libhyper.so (HyperPKI), libeToken.so (SafeNet), opensc-pkcs11.so (OpenSC — partial support for some tokens).
Linux driver stack: pcscd ← libpcsclite ← vendor PKCS#11 .so.
Install: sudo apt install pcscd pcsc-tools opensc

🔬 **[EXPERT] Expert — Deep Technical Detail**
PKCS#11 signing: C_Login(CKU_USER,PIN) → C_SignInit(CKM_SHA256_RSA_PKCS or CKM_ECDSA)
→ C_Sign(hash) — all inside the token. OpenSSL 3.x uses pkcs11-provider; legacy uses libpkcs11-helper or libengine-pkcs11-openssl. Key URI scheme:
-key 'pkcs11:token=<label>;object=<id>;type=private'
ePass2003 quirk: newer firmware (VID:PID 096e:080a) not recognised by OpenSC; use vendor libcastle.so.1.0.0. HYP2003: use HyperPKI libhyper.so (v230317).

13.2 OpenSSL + DSC Token Commands

```
# Setup (Ubuntu/Debian)
sudo apt install pcscd pcsc-tools opensc libengine-pkcs11-openssl
sudo systemctl enable --now pcscd

# Verify token detected
pcsc_scan
pkcs11-tool --module /path/to/vendor.so -L

# PKCS#11 module paths (common Indian tokens):
#  ePass2003/HYP2003: /usr/lib/x86_64-linux-gnu/libcastle.so.1.0.0
#  HyperPKI (HYP2003): /usr/lib/libhyper.so
#  SafeNet eToken:    /usr/lib/libeToken.so
#  OpenSC fallback:   /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so

# List objects on token (certificates and keys)
pkcs11-tool --module <vendor.so> --login --list-objects
```

```
# Export certificate from token (public — safe to export)
pkcs11-tool --module <vendor.so> --login --read-object \
    --type cert --label "Certificate" -o cert.der
openssl x509 -in cert.der -inform DER -out cert.pem
openssl x509 -in cert.pem -text -noout

# Sign a file using the private key ON the token (key never leaves)
openssl dgst -sha256 -engine pkcs11 -keyform engine \
    -sign 'pkcs11:token=<Label>;object=<KeyLabel>;type=private' \
    -out document.sig document.pdf

# Verify the signature (no token needed)
openssl dgst -sha256 -verify cert.pem -signature document.sig document.pdf

# Create CMS/PKCS#7 signed document using token
openssl cms -sign -binary -nodetach -engine pkcs11 \
    -inkey 'pkcs11:token=<Label>;object=<KeyLabel>;type=private' \
    -keyform engine -signer cert.pem \
    -in document.pdf -out document.p7s -outform DER

# Verify against India PKI chain
cat rcai.pem intermediate.pem > india_chain.pem
openssl verify -CAfile india_chain.pem cert.pem
openssl cms -verify -in document.p7s -inform DER -binary \
    -CAfile india_chain.pem -content document.pdf
```

Ch. 14

Aadhaar eSign

Server-side signing without a token — OTP, Biometric, Offline XML

14.1 What Is Aadhaar eSign?

🧒 **[ELI5] ELI5 — Explain Like I'm Five**

Aadhaar eSign lets you sign a document on your phone with just your Aadhaar number and an OTP. No USB stick, no special software. Trusted government servers sign it for you, then immediately delete the signing key. This is perfectly legal under the IT Act.

🌱 **[BEGINNER] Beginner — No Prior Knowledge**

Aadhaar eSign is a MeitY-backed service letting any Aadhaar holder sign documents digitally using mobile OTP or biometrics. Licensed eSign Service Providers (ESPs) handle the signing on secure HSMs. Providers: CDAC e-Hastakshar, eMudhra, Protean (NSDL), CDSL Ventures. Applications integrate via the CCA eSign API v3.2. The signing key is generated fresh, used once, and immediately destroyed. No USB token is required.

⚙️ **[INTERMEDIATE] Intermediate — Technical Background**

Flow: ASP sends eSign request XML (signed with ASP's own DSC) to ESP.
User authenticates via UIDAI: OTP to Aadhaar-linked mobile, biometric, or Offline XML.
ESP creates fresh key pair on HSM, issues short-lived X.509 cert (30 min validity), signs the document hash, destroys the key, returns signature + cert in response XML.
The document itself is never sent to the ESP — only its SHA-256 hash.
KYC modes: Online OTP, Online Biometric, Offline XML (strongest privacy).

🔬 **[EXPERT] Expert — Deep Technical Detail**

eSign API v3.2 request: <eSignRequest ver='3.1' sc='Y' ts='..' txn='..'
ekycIdType='A' ekycId='...' aspId='...' AuthMode='1' responseSigType='pkcs7'
Docs[InputHash+hashAlgorithm=SHA256]>
ASP signs request with its own CMS/DSC. Only SHA-256 hash of document is transmitted.
ESP sub-CA issues cert with: Subject={name, YOB, gender from Aadhaar eKYC}, no Aadhaar number.
Response: <eSignResponse><UserX509Certificate>...</UserX509Certificate>
<Signatures><DocSignature pkcs7Sig='...'></Signatures></eSignResponse>
Offline XML KYC: UIDAI Paperless eKYC ZIP — user provides signed XML + share code.
ASP verifies UIDAI signature on XML, extracts eKYC data offline without UIDAI API call.

Ch. 15

DigiLocker Integration

Document vault, OAuth2 API, verification, sharing

15.1 What Is DigiLocker?

🧒 **[ELI5] ELI5 — Explain Like I'm Five**

DigiLocker is like a government-approved safe on your phone. It holds your Aadhaar, PAN, driving licence, and other official documents. Banks and offices can look at documents directly from your DigiLocker — you do not need to carry paper copies or worry about forgeries.

🌱 **[BEGINNER] Beginner — No Prior Knowledge**

DigiLocker (digilocker.gov.in) is a MeitY initiative storing and sharing government-issued documents digitally. Over 6 billion documents from 2800+ issuers are accessible.
Documents stored in DigiLocker are legally equivalent to originals (IT Act §7A).
You can also eSign documents inside DigiLocker using Aadhaar OTP.
Applications access your documents via OAuth2 API with your explicit consent.

⚙️ **[INTERMEDIATE] Intermediate — Technical Background**

DigiLocker Partner API (OAuth2 + PKCE S256, apisetu.gov.in):
1. OAuth2 authorise → access token (scope: files.issued.r, files.uploaded.r, profile.r)
2. GET /1/files/issued — list government-issued documents (Aadhaar, PAN, DL, etc.)
3. GET /1/file/<uri> — fetch signed PDF or XML document
Documents arrive as digitally signed PDFs (PAdES/CAdES) or UIDAI-signed XML.
Access tokens expire; refresh via /oauth2/1/token with refresh_token grant.

🔬 **[EXPERT] Expert — Deep Technical Detail**

DigiLocker document signatures: CAdES-B or PAdES-LT, signed by DigiLocker signing certificate (issued under CCA-licensed CA). Verify: extract PKCS#7 from PDF /ByteRange, verify against DigiLocker signing cert, check chain to RCAI.
XML documents: verify XML-DSig <ds:Signature> using issuer cert in <KeyInfo>.
PKCE mandatory (S256). Production access requires MeitY onboarding + NDA via apisetu.gov.in.
Rate limits per client_id. Webhook events for document update/revocation.

Ch. 16

Government Portals

MCA21, GSTN, DGFT, Income Tax, EPFO, NeSL, eCourts

16.1 Portal Reference

Portal	Who needs DSC	DSC type	Accepts Aadhaar eSign?
MCA21 V3 (Company filing)	Company directors, CS, CA for ROC forms	Class 3 Sign (Individual + DIN)	Yes — for some forms (v3)
GSTN (GST)	Companies with paid-up capital ≥ ₹5L, certain taxpayers	Class 3 Sign (Individual or Org)	Yes — via ESP integration
Income Tax e-filing	Taxpayers with audit obligations, CA signing 3CA/3CB	Class 3 Sign (Individual + PAN)	No — DSC required for audits
DGFT (EXIM)	Importers, exporters for IEC and licence filing	Class 3 DGFT Sign (special OID)	Partial — check current DGFT advisory
EPFO Unified	Employers filing ECR, large PF withdrawals	Class 3 Sign (Individual)	No — token DSC required
NeSL (Legal docs)	Lenders, borrowers for loan agreements, SARFAESI	Class 3 Sign or Aadhaar eSign	Yes — core to NeSL platform
eCourts / Legal	Advocates, litigants for digital filing	Class 3 Sign	Partial — court-specific
Patents / Trademarks	Applicants, agents filing IP applications	Class 3 Sign	No — token DSC required

16.2 DSC Registration Flow (Common to All Portals)

9. Obtain Class 3 DSC from a licensed CA — video KYC, fee ₹800–₹2500/year.
10. Install vendor token driver (Windows: installer; Linux: libcastle.so or libhyper.so).
11. Plug in token, log in to portal, navigate to DSC Registration / Update DSC.
12. Portal reads certificate from token via browser plugin or Java applet.
13. Map DSC to your portal ID (DIN for MCA21, GSTIN for GSTN, PAN for IT).
14. Each filing: token must be plugged in; portal prompts for PIN at signing time.

Note: Most Indian government portals still require Internet Explorer compatibility mode or a Java signing applet on Windows. sslOpenCrypt's portal integration (Module 10) addresses this by providing a local signing agent that the browser extension calls — eliminating the Java dependency entirely.

Ch. 17

sslOpenCrypt India DSC Module

Module 10 specification — sub-modules, UI, and portal integration roadmap

17.1 Module 10 Sub-Modules

DSC Token Manager	Auto-detect USB tokens on hotplug. PKCS#11 library manager (pre-populated with Indian token paths). Certificate viewer. PIN management. On-token signing. CMS/PAdES output. Bulk sign for batch ITR/audit report workflows.
Aadhaar eSign Client	ESP selector (CDAC, eMudhra, Protean, CDSL). Auth mode: OTP / Biometric / Offline XML. Document hash sent — document stays local. ASP signing via token or key file. Response verification. Audit log. Developer sandbox mode with raw XML display.
DigiLocker Connect	OAuth2 + PKCE login. Document browser. Fetch + verify (signature chain to RCAI). eSign via DigiLocker (Aadhaar OTP). Share URI generation for KYC. Offline export + local signature verification.
India PKI Trust Store	RCAI + intermediate CA bundle — auto-refresh from cca.gov.in weekly. CRL cache. OCSP proxy with configurable TTL. Chain builder from any Indian DSC cert. RCAI fingerprint display for manual verification.
Portal Signing Agent	Local HTTP service (localhost only) that portal browser extensions call. Eliminates Java/ActiveX dependency for DSC signing on MCA21, GSTN, ITD. Returns signed hash or signed document per portal protocol.

17.2 Portal Integration Roadmap

Portal	Integration	Target
eCourts / manual sign	Sign PDF locally, upload manually	v1.0
DigiLocker	OAuth2 REST API — full integration	v1.2
MCA21 V3	Browser extension + local signing agent	v1.2
Income Tax e-filing	Browser extension + local agent	v1.2
NeSL	eSign API integration	v1.3
GSTN	Browser extension	v1.3
DGFT	Browser extension	v1.3
EPFO Unified	Browser extension	v1.3

Algorithm Quick-Reference

Symmetric Ciphers

Algorithm	Key bits	Status	Use for
AES-256-GCM	256	Current — preferred	TLS, file encryption, HTTPS — default choice
AES-128-GCM	128	Current — secure	TLS where 256-bit overhead matters
ChaCha20-Poly1305	256	Current — preferred	Mobile, no AES-NI hardware
AES-256-CBC	256	Legacy — add HMAC	Disk encryption, older TLS — new code use GCM
3DES	112	DEPRECATED (NIST 2023)	Legacy compatibility only — never new code
RC4	varies	BROKEN — never use	Do not use under any circumstances

Asymmetric Algorithms

Algorithm	Key bits	Security bits	Status
Ed25519	256	~128	Current — preferred for new applications
ECDSA P-256	256	~128	Current — FIPS environments, TLS, code signing
ECDSA P-384	384	~192	Current — high-security, government
RSA-4096	4096	~140	Good — CA keys, high-security RSA
RSA-3072	3072	~128	Acceptable (NIST through 2031) — minimum new RSA
RSA-2048	2048	~112	Legacy compatibility only — avoid for new work
ML-KEM-768	1184B pub	~160	FIPS 203 — post-quantum key exchange
ML-DSA-65	1952B pub	~160	FIPS 204 — post-quantum signatures
SLH-DSA	32–64B pub	~128–256	FIPS 205 — hash-based, most conservative PQC

Hash Algorithms

Algorithm	Output bits	Status	Use for

SHA-256	256	Secure — standard	General purpose — default
SHA-384	384	Secure	TLS, when 256-bit margin is insufficient
SHA-512	512	Secure	High-security, streaming
SHA-3-256	256	Secure — sponge	Where SHA-2 alternatives needed (PQC transition)
BLAKE2b	512	Secure — fast	File hashing, non-password keyed hashing
SHA-1	160	BROKEN (collisions)	Do not use
MD5	128	BROKEN	Do not use

Glossary

All terms used in sslOpenCrypt, defined precisely for practitioners at every level.

AEAD Authenticated Encryption with Associated Data. A cipher mode (AES-GCM, ChaCha20-Poly1305) providing confidentiality and integrity in one operation. The gold standard for symmetric encryption.

Argon2id Memory-hard password hashing function (RFC 9106). Resists GPU attacks by requiring large amounts of RAM. Use for key derivation from passwords. Current OWASP minimum: m=65536KiB, t=3, p=4.

ASN.1 Abstract Syntax Notation One. The data structure language used for X.509 certificates, PKCS formats, and most cryptographic data. DER is the binary encoding; PEM is DER in Base64.

CA Certificate Authority. Issues X.509 certificates binding a public key to an identity. Root CAs are pre-trusted in OS/browser root stores. In India: licensed by CCA under IT Act 2000.

CAdES CMS Advanced Electronic Signatures (ETSI EN 319 122). Enhances PKCS#7 with timestamps (CAdES-T), revocation data (CAdES-LT), and archive timestamps (CAdES-LTA) for long-term validity.

CCA Controller of Certifying Authorities. The Indian government body (under MeitY) that licenses CAs, operates the RCAI root, and sets policy for India PKI via CCA-CP and CCA-IVG guidelines.

CMS Cryptographic Message Syntax (RFC 5652). IETF standard for structuring signed and encrypted data. Basis for S/MIME and CAdES. Supersedes PKCS#7.

CRL Certificate Revocation List. A signed list of revoked certificate serial numbers published by a CA. Alternative to OCSP for offline revocation checking.

CSPRNG Cryptographically Secure Pseudo-Random Number Generator. OS-provided: /dev/urandom or getrandom() on Linux, CryptGenRandom on Windows. Never use rand() or time() for key generation.

DSC Digital Signature Certificate. In India: a Class 3 X.509 certificate issued by a CCA-licensed CA, stored on a FIPS 140-2 Level 2+ certified USB token. Contains PAN and optionally DIN or Aadhaar (masked) in SANs.

ECDSA Elliptic Curve Digital Signature Algorithm. Smaller keys than RSA at equivalent security (P-256 ≈ RSA-3072). Nonce reuse catastrophically leaks the private key — use Ed25519 which derives nonce deterministically.

Ed25519 Edwards-curve Digital Signature Algorithm on Curve25519 (RFC 8032). Deterministic nonce, fast, constant-time. The recommended signature algorithm for new applications.

eSign Aadhaar-based electronic signature service (MeitY, 2015; CCA API v3.2). Uses UIDAI eKYC for authentication; signing key generated on ESP HSM, used once, destroyed immediately. Legal under IT Act Section 3A.

ESP eSign Service Provider. A CCA-licensed entity (CDAC, eMudhra, Protean/NSDL, CDSL Ventures) that operates HSMs for Aadhaar eSign and issues short-lived signing certificates to authenticated Aadhaar holders.

GCM Galois/Counter Mode. An AEAD cipher mode combining CTR encryption with GHASH authentication. The standard for TLS 1.3 and modern file encryption. Nonce reuse is catastrophic — always use a random or counter nonce.

HKDF HMAC-based Key Derivation Function (RFC 5869). Two steps: Extract (salt+IKM→PRK) then Expand (PRK+info+L→OKM). Used in TLS 1.3 key schedule.

HMAC Hash-based Message Authentication Code. Keyed hash: HMAC(K,m)=H((K⊕opad)||H((K⊕ipad)||m)). Provides integrity and authentication. Does not provide non-repudiation — use digital signatures for that.

IVG Identity Verification Guidelines. CCA document specifying KYC requirements for DSC issuance. July 2024 update mandated video KYC and direct invoicing from CAs. Available at cca.gov.in.

ML-KEM Module Lattice Key Encapsulation Mechanism (FIPS 203). NIST post-quantum KEM standard, formerly Kyber. Replaces ECDH/RSA key exchange. Based on Module-LWE hardness assumption.

ML-DSA Module Lattice Digital Signature Algorithm (FIPS 204). NIST post-quantum signature standard, formerly Dilithium. Based on Module-LWE and Module-SIS. Replaces ECDSA/Ed25519 for PQC.

OCSP Online Certificate Status Protocol (RFC 6960). Real-time revocation checking for X.509 certificates. Preferred over CRL for live applications. OCSP stapling embeds the signed response in the TLS handshake.

PAdES PDF Advanced Electronic Signatures (ETSI EN 319 132). Embeds digital signatures in PDFs. Used for signing DigiLocker documents, legal contracts, and government portal forms.

PBKDF2 Password-Based Key Derivation Function 2 (RFC 8018). Applies HMAC-SHA-256 repeatedly to slow brute-force. Use ≥600,000 iterations (OWASP 2024). Less GPU-resistant than Argon2id.

PKCS#11 Cryptoki API standard for hardware security modules and smart cards. Allows OpenSSL and applications to use keys stored in USB tokens without exporting them. Used by all Indian DSC tokens.

PKCS#12 Personal Information Exchange format (.pfx/.p12). Bundles private key + certificate + chain in one encrypted file. Used for importing/exporting identities between systems (Thunderbird, Outlook, web servers).

PKI Public Key Infrastructure. The system of CAs, certificates, policies, and procedures enabling parties to verify each other's public keys through a chain of trust. India PKI is rooted at RCAI.

RCAI Root Certifying Authority of India. Operated by CCA/MeitY. The trust anchor for all Indian DSCs. Signs all licensed CA certificates. RCAI certificate available at cca.gov.in/repository.

RSA Rivest-Shamir-Adleman. First widely-used public-key system; based on difficulty of factoring large integers. Use OAEP for encryption and PSS for signatures. Slower and larger keys than ECC; being superseded.

SAN Subject Alternative Name. X.509 v3 extension listing additional identities: DNS names, IP addresses, email addresses. Required for modern TLS certificates. Indian DSCs use SAN for PAN, DIN, Aadhaar (masked).

SHA-256 Secure Hash Algorithm 256-bit. SHA-2 family. The standard hash for digital signatures, TLS, and file integrity in India PKI. Mandatory for all Indian DSC signatures since SHA-1 deprecation.

SLH-DSA Stateless Hash-Based Digital Signature Algorithm (FIPS 205). Post-quantum signature scheme based solely on hash function security — the most conservative PQC choice. Larger signatures (8–50 KB) than ML-DSA.

S/MIME Secure/Multipurpose Internet Mail Extensions (RFC 8551). Standard for signing and encrypting email using CMS and X.509 certificates. Supported natively by Outlook, Thunderbird, and Apple Mail.

TLS Transport Layer Security. Cryptographic protocol securing HTTPS, email, VPN. TLS 1.3 (RFC 8446, 2018) is the current standard. Mandatory forward secrecy; three AEAD-only cipher suites.

UIDAI Unique Identification Authority of India. Issues Aadhaar numbers. Provides eKYC API (OTP/biometric/offline XML) used by eSign service providers for Aadhaar-based identity verification.

X.509 ITU-T standard defining the format of public key certificates and CRLs. Version 3 adds extensions for SANs, key usage, CA constraints, and policy OIDs. Used by all Indian DSCs and TLS certificates.

X25519 Elliptic curve Diffie-Hellman on Curve25519 (RFC 7748). Used for key agreement in TLS 1.3 and SSH. Fast and constant-time. Not for signing — use Ed25519 for signatures.

India Certifying Authorities

All CAs are licensed by the Controller of Certifying Authorities (CCA) under the IT Act 2000. The authoritative and current list is at cca.gov.in/licensed_ca.html. Contact details and disclosure records are on each CA's website.

CA Name	Website	Key products	Notes
eMudhra Limited	emudhra.com	Class 3 Sign/Encrypt, DGFT, Document Signer, eSign	Largest CA by volume. eMudhra emSigner widely used.
(n)Code Solutions (GNFC)	ncodesolutions.com	Class 3 Sign/Encrypt, DGFT	Gujarat government CA. Widely used for MCA, IT filings.
Capricorn CA	capricornca.com	Class 3 Sign/Encrypt, DGFT	Strong in banking and insurance sectors.
XtraTrust	xtratrust.com	Class 3 Sign/Encrypt	Fast issuance; Aadhaar eKYC based.
SafeScrypt (Sify)	safescrypt.com	Class 3 Sign/Encrypt, SSL	Sify Technologies subsidiary.
CDAC (e-Hastakshar)	esign.cdac.in	eSign ESP only (not DSC token)	Government ESP; public eSign portal at esign.cdac.in.
Protean eGov (NSDL)	protean-tinpan.com	Class 3 DSC, eSignPro, PAN services	Heavy use in IT, TDS, mutual fund KYC.
CDSL Ventures	cvlindia.com	Class 3 DSC, eSign ESP	Capital markets focus; depository participant use.
Verasys Technologies	verasys.in	Class 3 Sign/Encrypt	Aadhaar eKYC based issuance.
TCS CA	tcs.com	Class 3 (enterprise)	Primarily for TCS client deployments.

APPENDIX D

Pen Drive Contents

The pen drive included with this book contains everything needed to install and run sslOpenCrypt on Linux, macOS, and Windows — without requiring an internet connection.

sslOpenCrypt AppImage (Linux)	sslopencrypt-x.x.x-x86_64.AppImage — runs without installation on any modern x86_64 Linux. chmod +x, then double-click or ./sslopencrypt-...AppImage. No root required.
sslOpenCrypt .dmg (macOS)	sslopencrypt-x.x.x.dmg — drag to Applications. Requires macOS 12+ (Monterey or later). Gatekeeper: right-click → Open on first launch.
sslOpenCrypt installer (Windows)	sslopencrypt-x.x.x-setup.exe — NSIS installer. Requires Windows 10/11 x64. Includes bundled OpenSSL 3.x if system OpenSSL is not found.
OpenSSL binaries (Windows)	openssl-3.x.x-win64/ — standalone openssl.exe for Windows, in case system PATH does not include OpenSSL. Used by sslOpenCrypt automatically.
Source code	sslopencrypt-x.x.x-source.tar.gz — full GPL v3 source. Build instructions in README.md.
India PKI trust chain	india_pki/ — RCAI root certificate + all active intermediate CA certs downloaded from cca.gov.in/repository at time of pressing. Refresh from CCA for production use.
ePass2003 Linux driver	tokens/libcastle.so.1.0.0 — Feitian ePass2003 PKCS#11 library for Linux x86_64.
HYP2003 Linux driver	tokens/libhyper.so — HyperPKI middleware for HYP2003 token on Linux x86_64.
OQS-Provider (experimental)	oqs-provider/ — pre-built OpenSSL 3.x OQS provider for post-quantum algorithm experimentation. Linux and Windows. See Chapter 11.
This book (PDF)	sslOpenCrypt_Complete_Guide.pdf — searchable PDF of this printed book.
Quick-start card	quickstart.pdf — one-page getting-started guide for each platform.
Autorun (Windows)	autorun.inf + start.bat — opens the quick-start HTML on Windows Explorer open.

🌱 **[BEGINNER] Beginner — No Prior Knowledge**
Linux: open a terminal in the pen drive folder, type:
chmod +x sslopencrypt-*.AppImage && ./sslopencrypt-*.AppImage
macOS: double-click the .dmg, drag to Applications, right-click → Open.
Windows: double-click sslopencrypt-*-setup.exe and follow the installer.